Design Procedure

In this lab, our final task was to create an animation using the VGA Framebuffer that was provided for us. To complete this task, we first had to implement a line-drawing module that will accurately draw lines where we want on the screen. This was implemented using the Bresenham line-drawing algorithm. To create the animation, we followed the code outlined in the spec and developed our own error and y step calculations in the module. Finally we used the main module to draw the lines related to star one at a time with half a second pauses between each of the lines being drawn.

Task #1 N/A

Task #2

Drawing lines using the VGA framebuffer requires a Bresenham line-drawing algorithm. This algorithm is useful when handling steep slopes, when the Y axis changes faster than the X. Also, the framebuffer prefers to draw from the top left to the bottom right and from a smaller X to a larger X. These are all constraints that we implemented in our algorithm. Our ASMD for this module is given in figure 1 below and takes in 4 inputs (x0,y0,x1,y1) which determine the starting and ending points of our line. The module will cycle through the states deciding whether the line is steep or not and then precede on drawing it. Once the line is drawn, it will stay in the done state until reset is asserted.

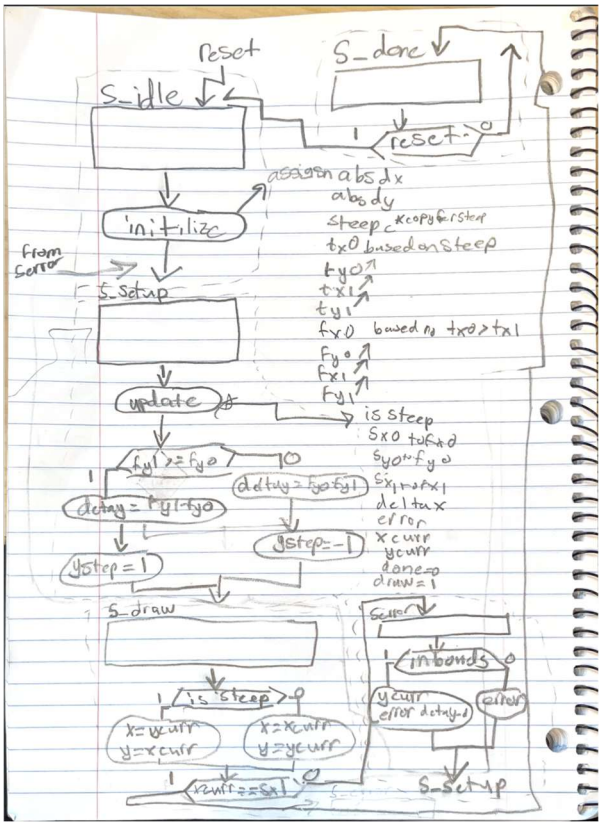

Figure 1: ASMD Chart for Line drawing algorithm

Task #3

For task 3, our objective was to demonstrate the working line_drawer module in each direction through an animation. We decided to animate a start by drawing each line of the star individually , waiting a little bit between each line, and then waiting some more after the whole star is drawn before clearing the screen and repeating the process. This was possible because each line on the star goes in a unique direction that is either steep or gradual. The star animation is demonstrated in Figure 11 below.

Overall System: Top Level Block Diagram

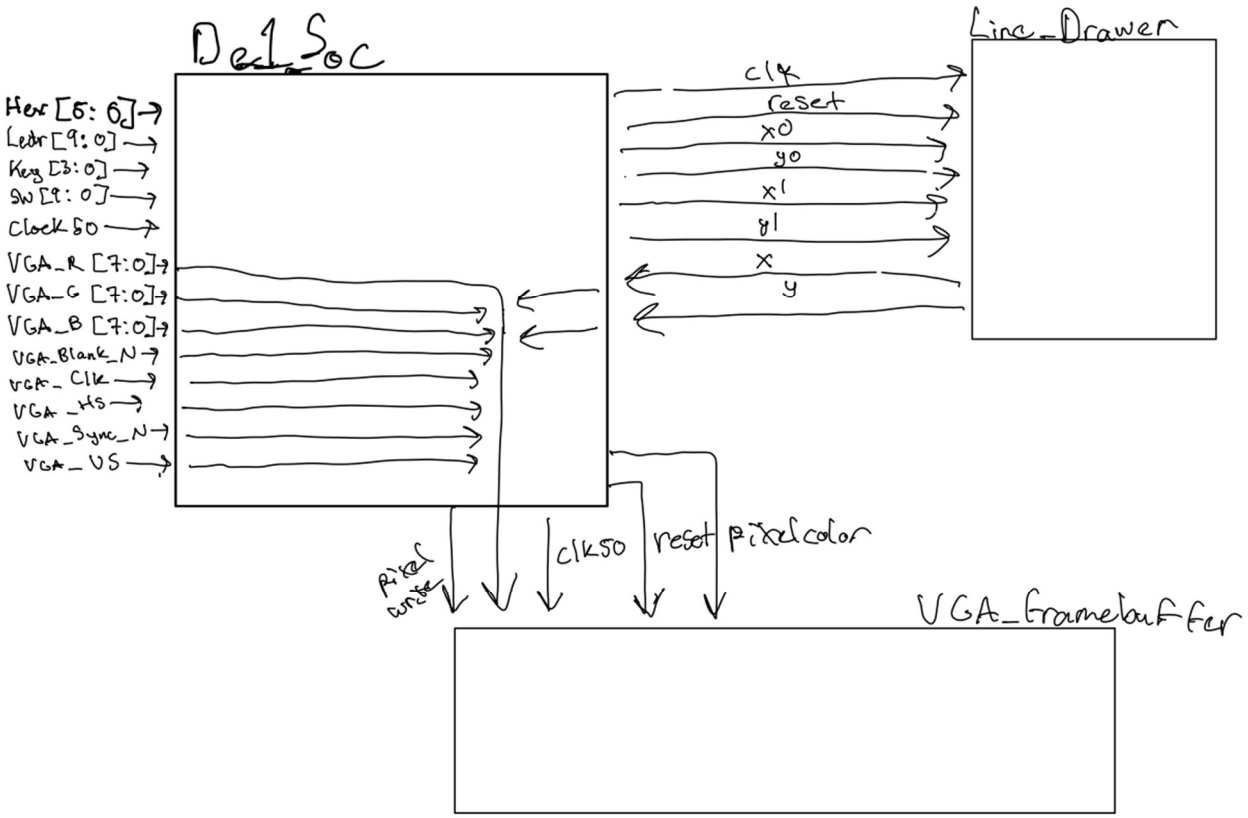

Figure 2: Top Level Block Diagram

Results

This module draws a a star over time and then clears and redraws. The animation of this star uses the DE1 as the main area for storing cords and then calls linedrawer to draw those lines. Finally, the framebuffer is putting the pixels to the screen. It pauses slightly less than half a second before each line. After the star is fully drawn the animation restarts.

Line_Drawer

The modelsim in Figure X-Y show the simulation for all cases of our Line Drawer module. It works by using our alogrthim and going from the smaller x cord between the start and ending point. X0 and y0 are our start x1 and y1 is the end. It is important to note that in cases 4 and 6 it appears x is derc but that is because it is steep so there is a swap in the x cords and y cords. Basically, x_curr always increments to N regardless of case it is just for some steep cases the direction swap is not visible but is for others. We also can see in all

cases that error is updating so that our equation is satisfied. Error works by checking how far the ideal line has drifted from the pixel it is on. We use deltax as the centers rounding so there is no bias up or down. Next we update y curr and the error and check next if it should be changed. Finally, once the x and y are in the specified location (start to end) done is asserted with bost ps and ns going to S_DONE.

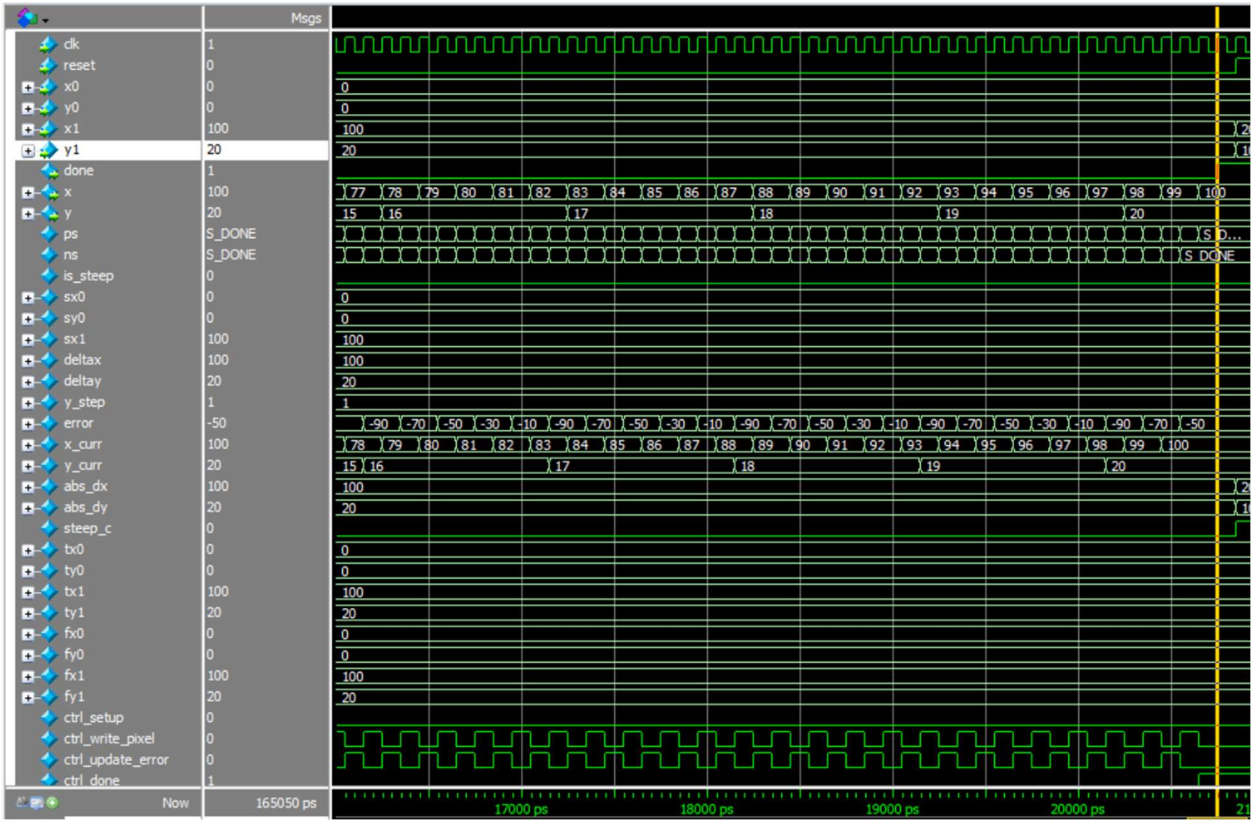

Figure 3: Case 1: Down Right Gradual

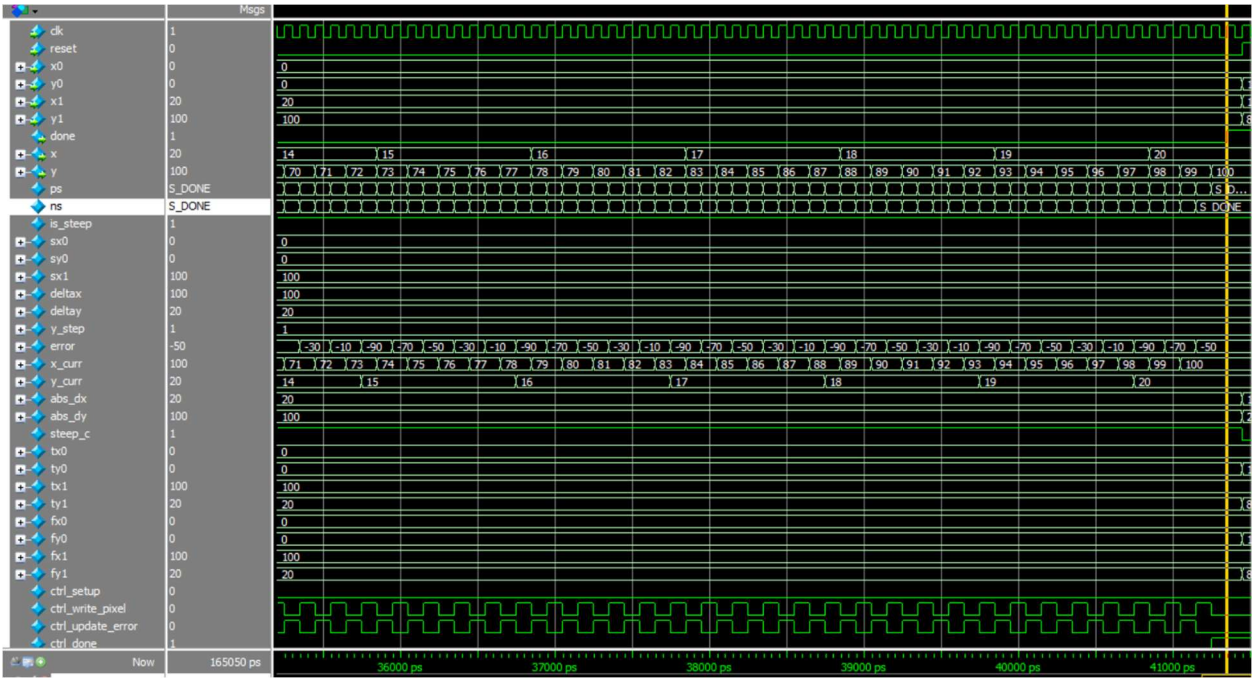

Figure 4: Case 2: Down Right Steep

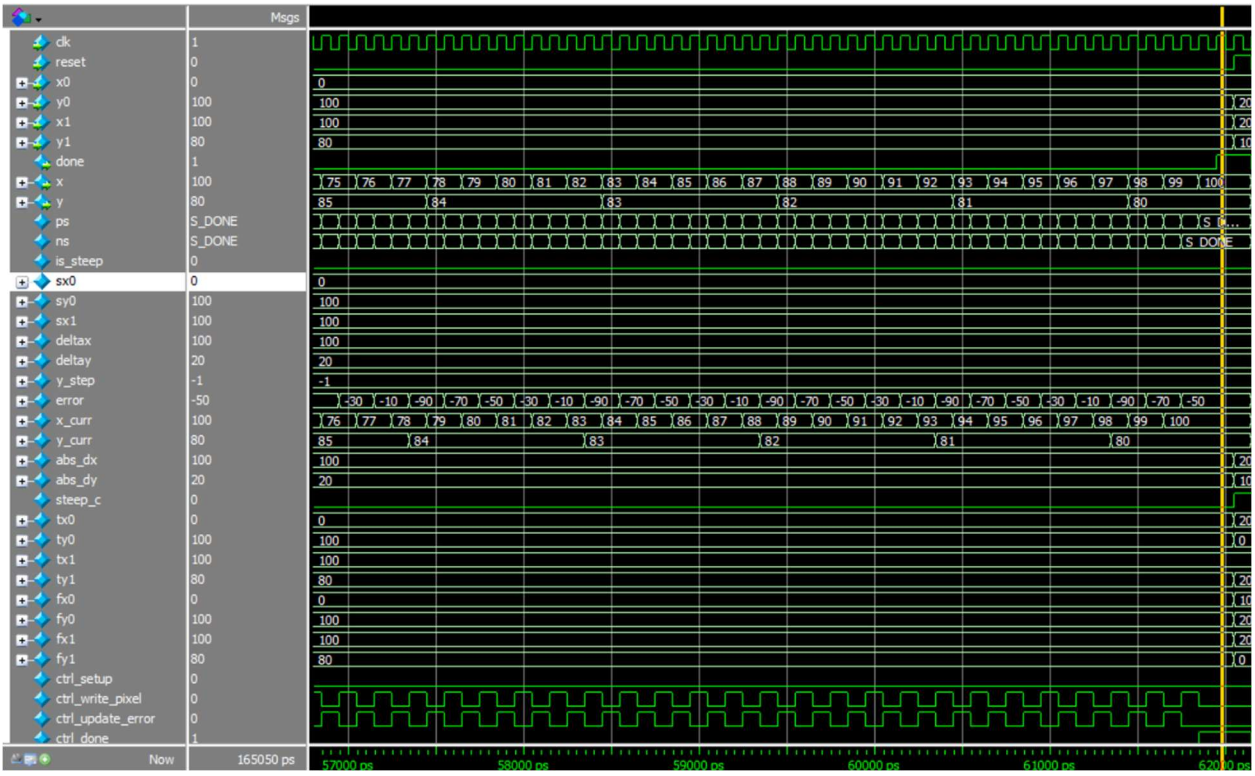

Figure 5: Case 3: Up Right Gradual

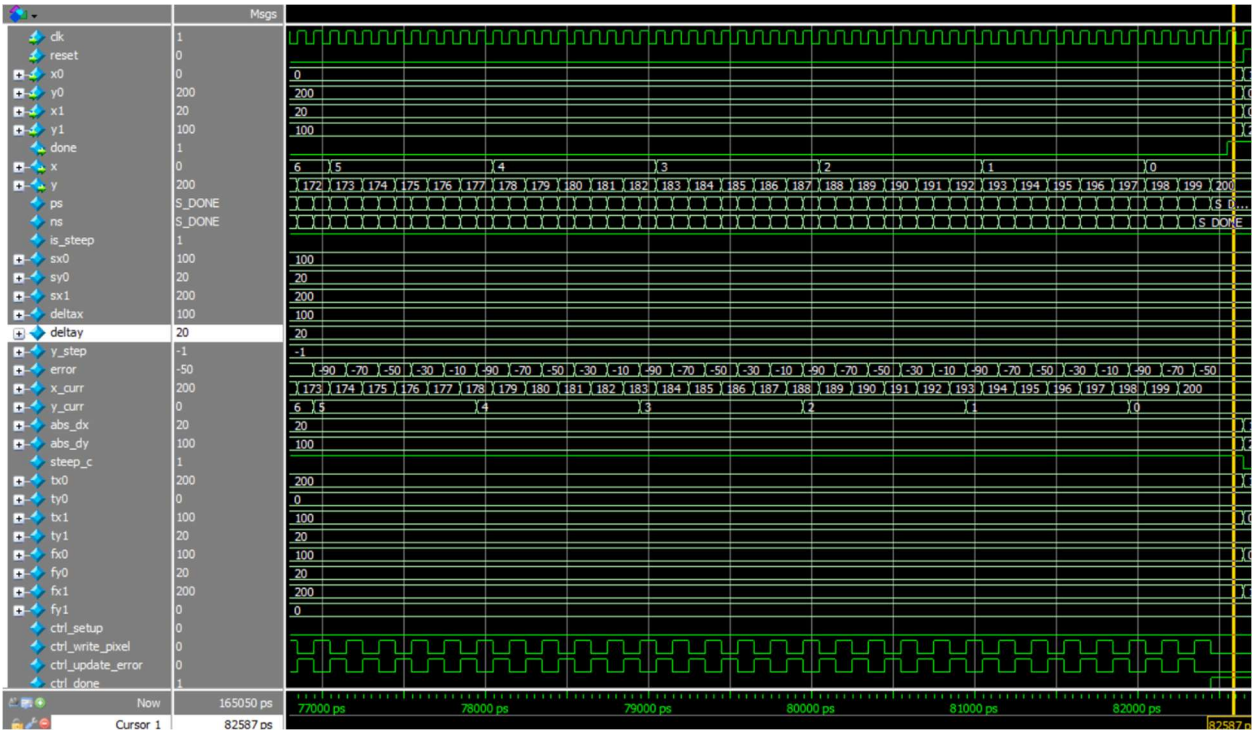

Figure 6: Case 4: Up Right Steep

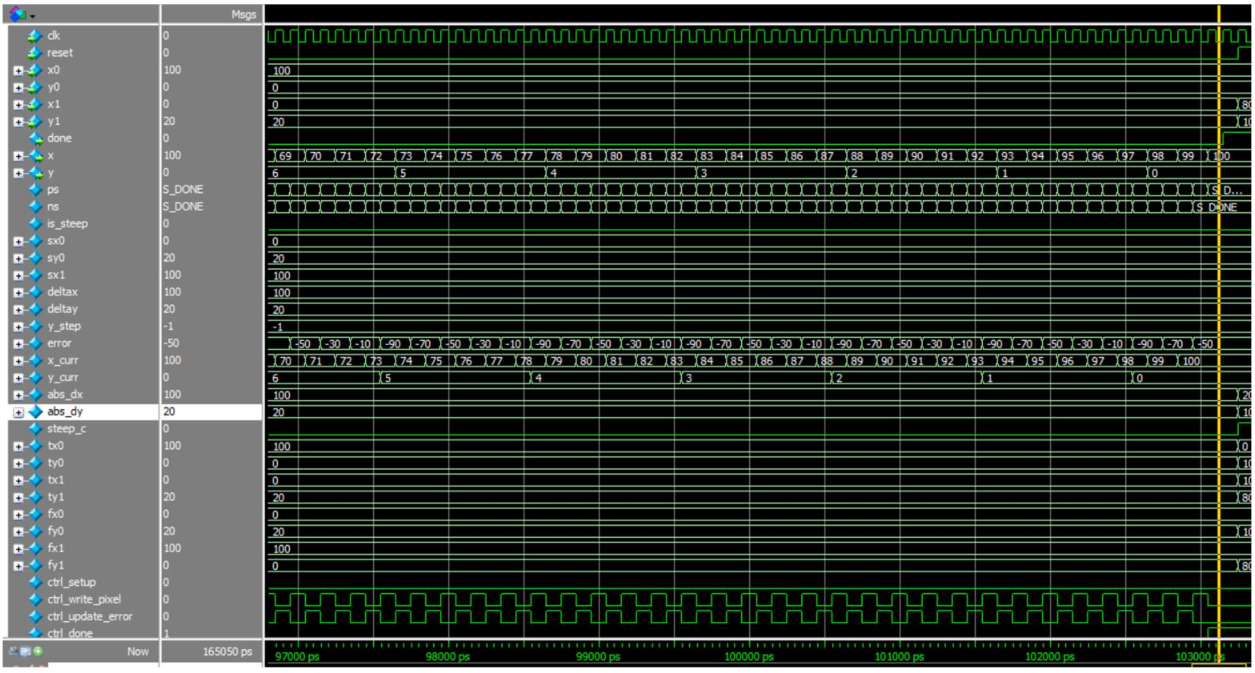

Figure 7: Case 5: Down Left Gradual

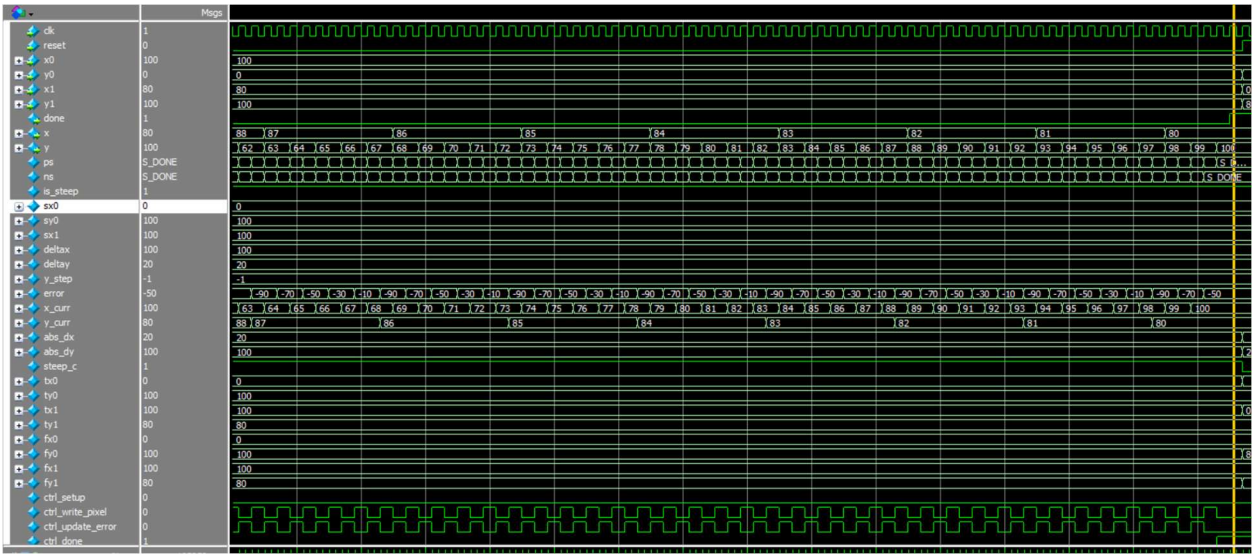

Figure 8: Case 6: Down Left Steep

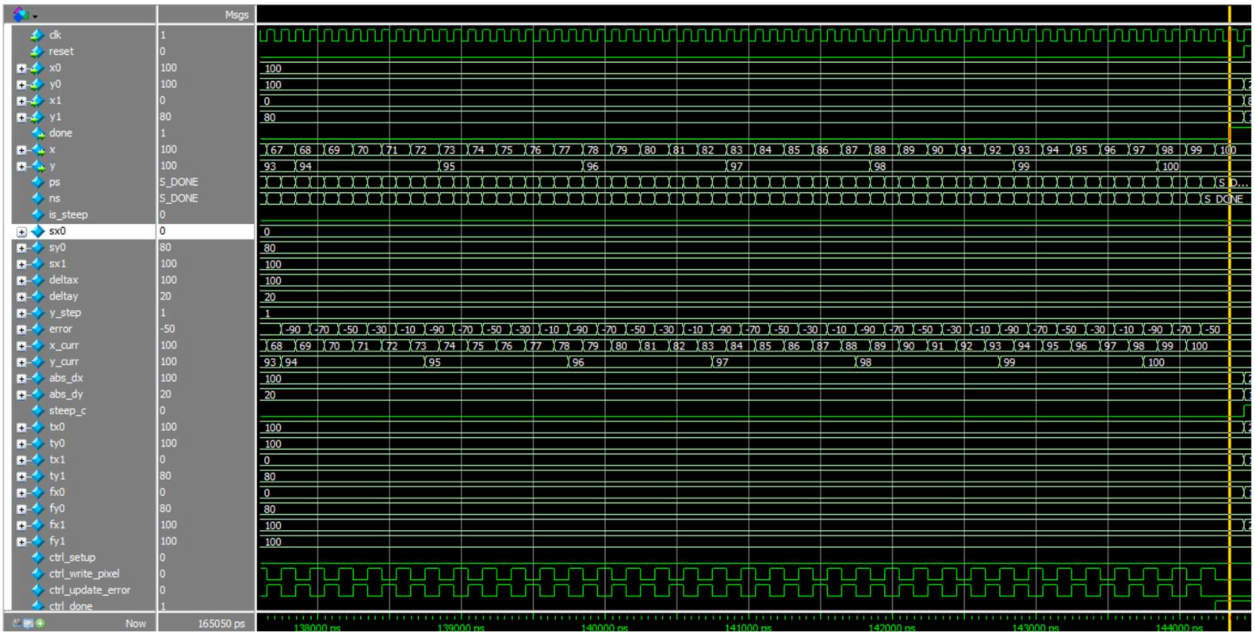

Figure 9: Case 7: Up Left Gradual

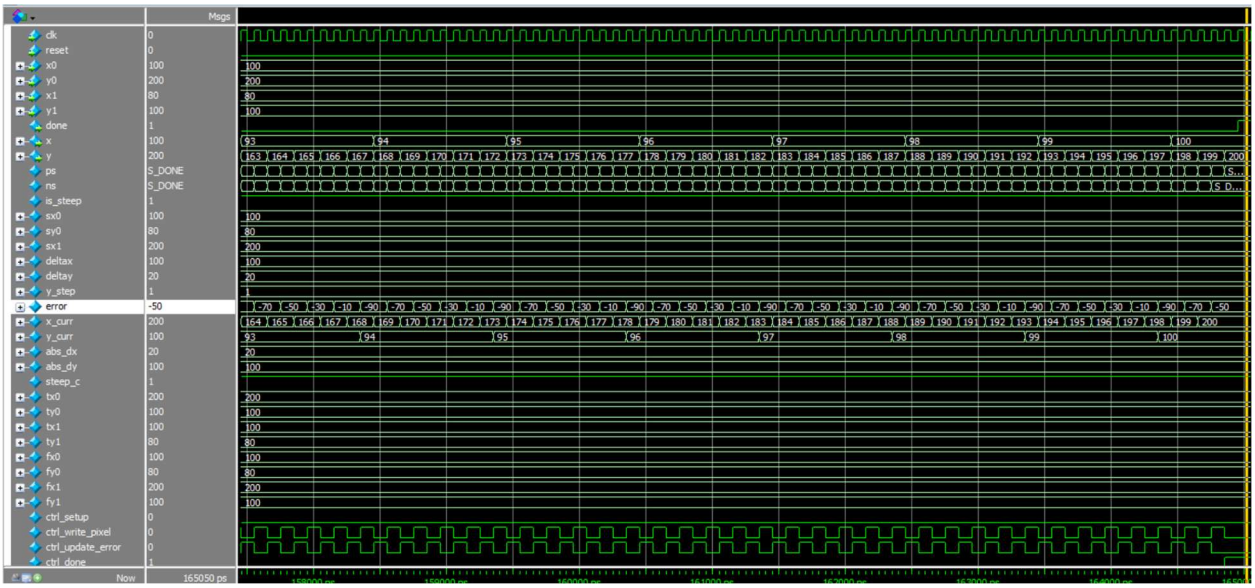

Figure 10: Case 8: Up Right Steep

DE1_SoC

The top-level module, DE1SoC, contains an FSM that connects and communicates to a data path the draws the lines using the linedrawer module. It also controls how long each line should be displayed before drawing a new line and how long the star should be displayed before clearing. This is through the INTERLINEPAUSE, and PAUSELAST parameters that we declared. While the VGAFramebuffer and linedrawer modules work on their own, the top level ensures that they work together to draw the star properly. When the reset signal is high, the FSM goes to the SClear state that will reset the clear x and clear y counters back to zero to ensure a clean black starting point. During the clear phase, the clear x and clear y counters increment to their respective dimensions to color each pixel to black precisely. The pixel write and pixel color inputs into the VGA determine what color the screen should be at each specific pixel and if it should color in the pixel or not. Think of the pixel write signal as a write enable. The SWait state controls the time after each line is drawn so that the registers in linedrawer do not overwrite eachother. The SDraw state is the state that controls the drawing of the lines, choosing which line to draw and incrementing between them. After each individual line is drawn, the state shifts to the SLineP which adds a delay between each line so that it is visible to the human eye, roughly 0.5 seconds using the pausecnt timer. Finally, once all 9 lines are drawn, the FSM transitions to S_Pause that will hold the star in the current state for 1.5 seconds until it begins clearing. The animation is modeled in figure 11 below.

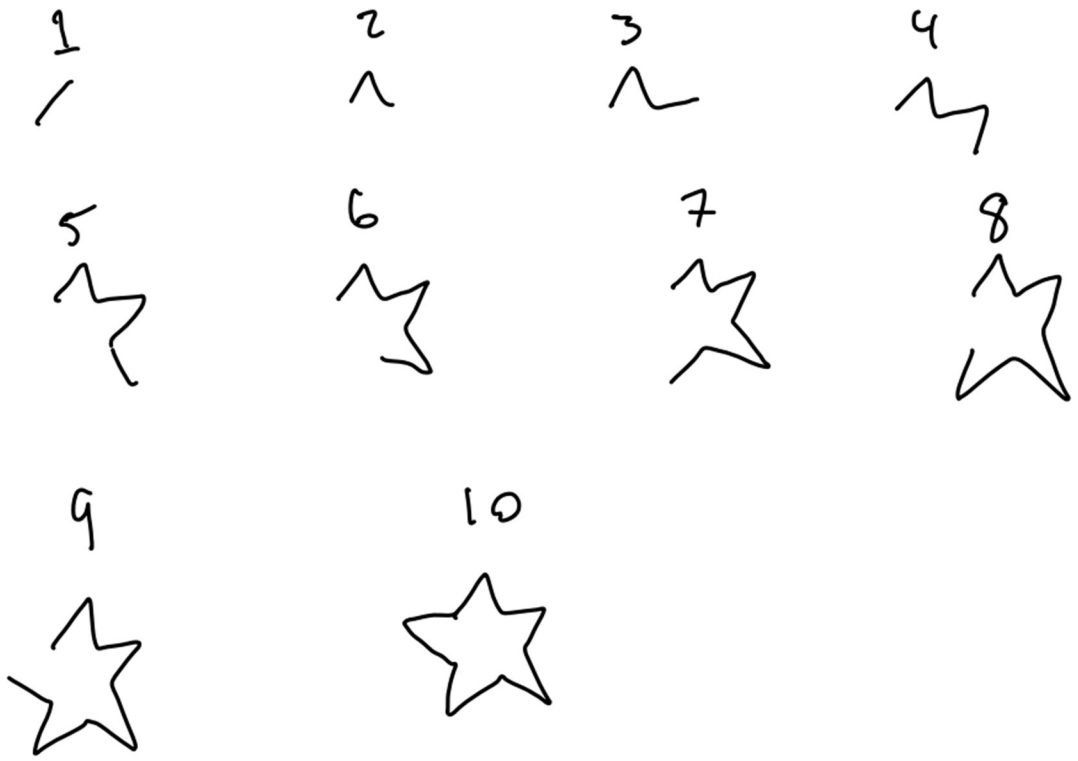

Figure 11: Drawing of the star animation

Flow Summary

Flow Summary

<<Filter>>

Flow Status	Successful - Fri May 22 12:51:08 2026
Quartus Prime Version	17.0.0 Build 595 04/25/2017 SJ Lite Edition
Revision Name	DE1_SoC
Top-level Entity Name	DE1_SoC
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	N/A
Total registers	171
Total pins	96
Total virtual pins	0
Total block memory bits	307,200
Total DSP Blocks	0
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0
Total DLLs	0

Figure 12: Flow Summary

Experience Report

We found this lab to be the hardest thing we have ever done in ECE. There were many many points we wanted to give up. Countless more times we needed help and even more times than that we had no clue what to do. Line Drawer was very hard to understand the algorithm and if the code hadn't been explained/ given in the spec we likely would have worked on it longer. Thankfully, once we had an idea of how the algorithm worked it snowballed into a working line drawer; it took several more hours to get to that point. Task 3 was one of the most frustrating. We ended up drawing a star, but we started with a smiley face which just wasn't working. We also wanted to use a MIF but ended up adding a counter instead and local variables.

This lab took us approximately 31 hours, broken down as follows:

- Reading – 1hr
- Planning – 3hrs
- Design – 7hrs
- Coding – 10hrs
- Testing – 5hrs
- Debugging – 5hrs